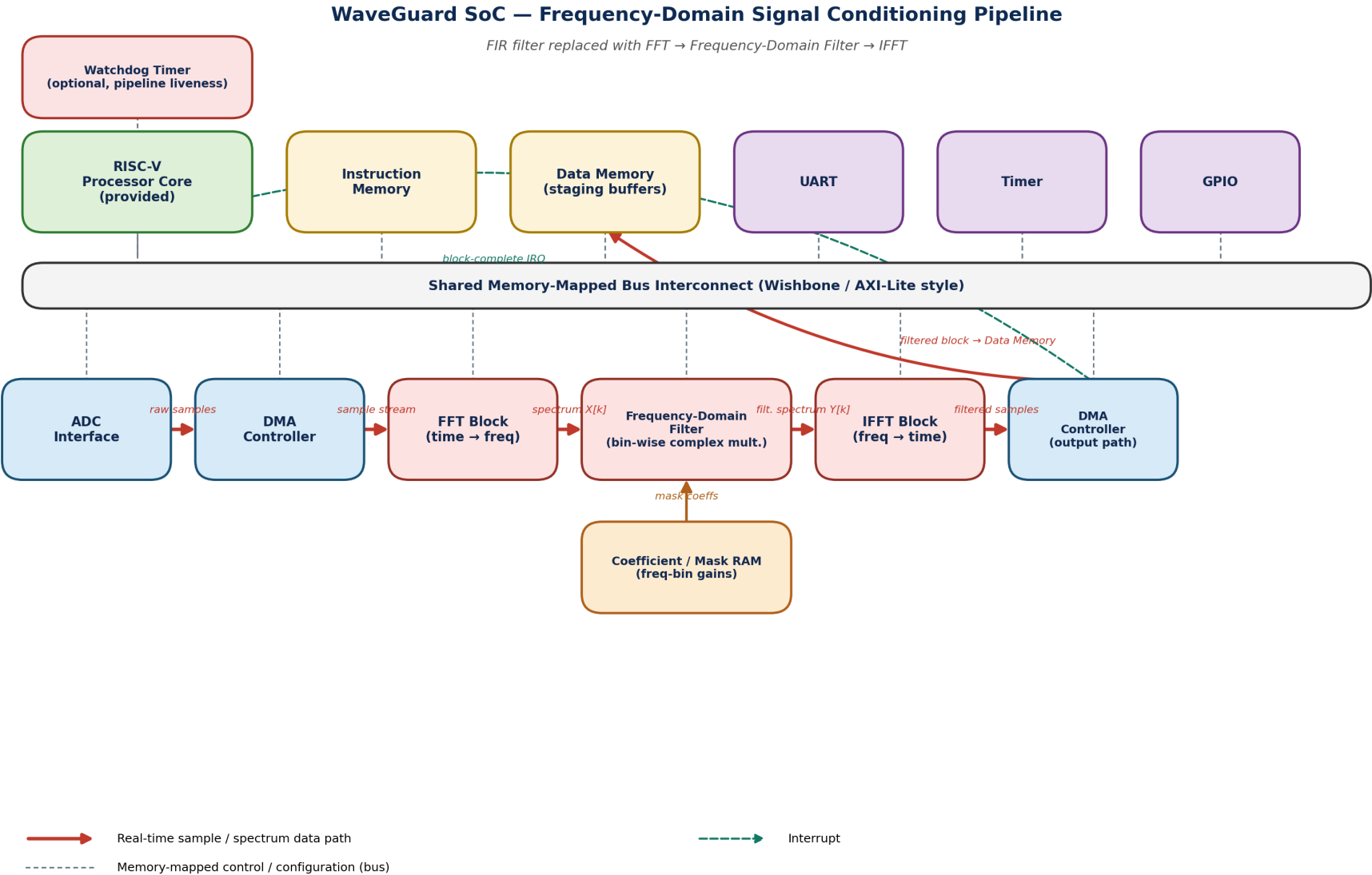


WaveGuard: A RISC-V-Based System-on-Chip with a Hardware FIR Filtering Pipeline for Real-Time Signal Conditioning

Revision: FIR filter block removed and replaced with a frequency-domain conditioning chain — FFT → Frequency-Domain Filter → IFFT. Diagram shows the full block architecture, signal flow graph, and the associated signal list.



Signal List — Frequency-Domain Data Pipeline

Signal	Source → Destination	Direction	Width	Domain	Description
adc_sample_data	ADC Interface → DMA Controller	Out	16b	Time	Latest raw ADC conversion result, updated each sample period
adc_sample_valid	ADC Interface → DMA Controller	Out	1b	Time	Pulses high for one cycle when adc_sample_data is valid
dma_req / dma_ack	DMA Controller ↔ ADC Interface	In/Out	1b each	Control	Handshake for DMA to pull each new sample
fft_in_data	DMA Controller → FFT Block	Out	16b	Time	Windowed time-domain sample streamed into the FFT input buffer
fft_in_valid / fft_in_ready	DMA Controller ↔ FFT Block	In/Out	1b each	Control	Streaming handshake for loading an N-point input frame
fft_out_re[k], fft_out_im[k]	FFT Block → Frequency-Domain Filter	Out	18b each	Freq	Real/imaginary parts of complex spectral bin k, k = 0..N-1
fft_out_valid	FFT Block → Frequency-Domain Filter	Out	1b	Freq	Asserted while a completed spectral frame is being read out
fft_busy / fft_done	FFT Block → Bus / IRQ logic	Out	1b each	Control	Status flags; fft_done can optionally raise an interrupt
mask_addr	Frequency-Domain Filter → Coefficient/Mask RAM	Out	log2(N) b	Freq	Index of the frequency bin whose gain is being fetched
mask_gain_re[k], mask_gain_im[k]	Coefficient/Mask RAM → Frequency-Domain Filter	Out	16b each	Freq	Per-bin complex gain (magnitude/phase mask) applied via complex multiply
filt_out_re[k], filt_out_im[k]	Frequency-Domain Filter → IFFT Block	Out	18b each	Freq	Filtered complex spectrum $Y[k] = X[k] \times \text{Mask}[k]$
filt_out_valid	Frequency-Domain Filter → IFFT Block	Out	1b	Freq	Frame-valid strobe for the filtered spectrum
ifft_out_data	IFFT Block → DMA Controller	Out	16b	Time	Reconstructed time-domain filtered sample
ifft_out_valid / ifft_out_ready	IFFT Block ↔ DMA Controller	In/Out	1b each	Control	Streaming handshake for the output block transfer
ifft_done / block_irq	IFFT Block → RISC-V Core (via bus IRQ line)	Out	1b	Control	Interrupt raised once a full filtered block has been written to Data Memory
dma_dest_addr, dma_len	RISC-V Core → DMA Controller	Out	32b, 16b	Control	Configured by firmware: destination base address and transfer length in Data Memory

Signal List — Bus, Memory, and Peripheral Interfaces

Signal	Source → Destination	Direction	Width	Description
bus_addr	RISC-V Core → Bus Interconnect	Out	32b	Memory-mapped address for register/memory access
bus_wdata / bus_rdata	RISC-V Core ↔ Bus Interconnect	In/Out	32b each	Write / read data for peripheral and accelerator registers
bus_we / bus_re	RISC-V Core → Bus Interconnect	Out	1b each	Write-enable / read-enable strobes
bus_sel[block]	Bus Interconnect → each peripheral/IP	Out	1b	Chip-select decoded from bus_addr for the target block
bus_ready / bus_err	Peripheral/IP → Bus Interconnect	Out	1b each	Transaction completion and error signaling
fft_ctrl, fdf_ctrl, ifft_ctrl	RISC-V Core → respective block (via bus)	Out	reg	Memory-mapped control/status registers: start, block size, enable
timer_tick	Timer → ADC Interface	Out	1b	Periodic pulse setting the ADC sampling cadence
uart_tx_data / uart_tx_valid	Data Memory (via core) → UART	Out	8b / 1b	Filtered output block streamed out serially
gpio_status[7:0]	RISC-V Core / pipeline blocks → GPIO	Out	8b	Debug/status bits: pipeline stage active, error flags
wdt_kick	RISC-V Core → Watchdog Timer	Out	1b	Firmware heartbeat; absence triggers a reset if a stage stalls
sys_clk / sys_rst_n	Clock/Reset source → all blocks	Out	1b each	Global synchronous clock (50 MHz) and active-low reset — see clock/timing spec

Clock and Sample-Rate Specification

All blocks share a single synchronous clock domain (sys_clk); no clock-domain-crossing logic is required. Sample-rate and bin-spacing values below assume the default ADC rate and are firmware-configurable within the stated range.

Parameter	Symbol	Value	Notes
System clock frequency	f_sys_clk	50 MHz	Drives RISC-V core, bus interconnect, and all peripheral/accelerator logic
ADC sampling frequency (default)	f_s	8 kHz	Set by timer_tick; linear (not log) sample-rate scale — suited to biomedical/audio bandwidths
ADC sampling frequency (configurable range)	f_s	1 kHz – 48 kHz	Range supported by the Timer prescaler; firmware-selectable at boot or runtime
FFT / IFFT block size	N	256 points	Sets frequency-bin resolution: $\Delta f = f_s / N$
Frequency-bin resolution (at default f_s)	Δf	31.25 Hz	$\Delta f = f_s / N = 8000 \text{ Hz} / 256$; bins are linearly spaced, not logarithmic
Nyquist / max representable frequency	f_s / 2	4 kHz (at default f_s)	Upper edge of the frequency-domain filter's usable mask range
UART baud rate	—	115200 baud	Independent asynchronous rate for streaming filtered output off-chip

Note on axis scale: the FFT/IFFT pipeline and the frequency-domain filter's coefficient mask operate on a linear frequency axis (uniformly spaced bins of width $\Delta f = f_s / N$), not a logarithmic one. mask_addr indexes bins linearly from 0 to N–1. A log-frequency axis is only a display convention for plotting a magnitude response and is not how bins are addressed in hardware.

FFT Algorithm Selection: DIT vs. DIF

The FFT and IFFT blocks both use the radix-2 **Decimation-in-Time (DIT)** algorithm, not Decimation-in-Frequency (DIF).

DIT produces its output in natural bin order ($k = 0..N-1$) from bit-reversed input order. This matches how the pipeline is already specified: `fft_out_re[k]` / `fft_out_im[k]` and `mask_addr` both index bins in natural order, and the Frequency-Domain Filter multiplies bin-by-bin against `mask_gain[k]` in that same order — so no reorder stage is needed between the FFT and the filter. The same reasoning applies to the IFFT: it consumes the (naturally-ordered) filtered spectrum and must produce `ifft_out_data` in natural time order for the DMA to stream out, which is exactly DIT's output property.

DIT's one requirement — bit-reversed input order — is satisfied at no extra multiply-accumulate cost: the DMA writes streamed samples into the FFT input buffer using bit-reversed write addresses (an address permutation only), and likewise writes the filtered spectrum into the IFFT input buffer at bit-reversed addresses. Choosing DIF instead would have pushed an equivalent reorder onto the filter stage and required `mask_addr` to be indexed in bit-reversed order, conflicting with the signal list as already documented.